Topic 3: Variables, Data Types, and Constants

2.1 What is a Variable?

- A variable is a named storage location in memory that holds a value
- Think of it as a labeled box: the label is the name, the content is the value
- In C++ you must declare the type before using a variable: `int age = 20;`
- The type tells the compiler how much memory to reserve and how to interpret the bits
- Variable names: letters, digits, underscore; cannot start with a digit; no spaces; case-sensitive

2.2 Fundamental Data Types

- `int` : whole numbers, 4 bytes — `int score = 95;`
- `float` : decimal numbers, 4 bytes, ~7 significant digits — `float price = 9.99f;`
- `double` : decimal numbers, 8 bytes, ~15 significant digits — `double pi = 3.14159265;`
- `char` : single character, 1 byte — `char grade = 'A';`
- `bool` : true or false, 1 byte — `bool passed = true;`
- `string` : text (from `<string>` header) — `string name = "Ahmad";`

2.3 Constants

- `const int MAX = 100;` — value cannot change after initialization
- `#define PI 3.14159` — preprocessor replaces PI with 3.14159 throughout the code
- `const` is preferred over `#define` because it has a type and respects scope
- Convention: write constant names in UPPER_CASE

2.4 Type Sizes and Conversion

- `sizeof(int)` returns how many bytes that type uses on the current system
- Implicit conversion: `int x = 5; double d = x;` — automatically widened
- Explicit cast: `double d = 9.99; int i = (int)d;` → `i = 9` (decimal truncated)
- Integer overflow: if an int exceeds its range, it wraps around silently — be aware

2.5 Memory Model

- Every variable occupies a specific address in RAM

- `int x = 5;` reserves 4 bytes and stores the value 5 at that address
- The address can be retrieved with the address-of operator `&x`
- Understanding this prepares you for pointers later
